Regression Models for Count Data in R

Achim Zeileis
Universität Innsbruck

Christian Kleiber
Universität Basel

Simon Jackman
Stanford University

Abstract

The classical Poisson, geometric and negative binomial regression models for count data belong to the family of generalized linear models and are available at the core of the statistics toolbox in the R system for statistical computing. After reviewing the conceptual and computational features of these methods, a new implementation of hurdle and zero-inflated regression models in the functions `hurdle()` and `zeroinfl()` from the package **pscl** is introduced. It re-uses design and functionality of the basic R functions just as the underlying conceptual tools extend the classical models. Both hurdle and zero-inflated model, are able to incorporate over-dispersion and excess zeros—two problems that typically occur in count data sets in economics and the social sciences—better than their classical counterparts. Using cross-section data on the demand for medical care, it is illustrated how the classical as well as the zero-augmented models can be fitted, inspected and tested in practice.

Keywords: GLM, Poisson model, negative binomial model, hurdle model, zero-inflated model.

1. Introduction

Modeling count variables is a common task in economics and the social sciences. The classical Poisson regression model for count data is often of limited use in these disciplines because empirical count data sets typically exhibit over-dispersion and/or an excess number of zeros. The former issue can be addressed by extending the plain Poisson regression model in various directions: e.g., using sandwich covariances or estimating an additional dispersion parameter (in a so-called quasi-Poisson model). Another more formal way is to use a negative binomial (NB) regression. All of these models belong to the family of generalized linear models (GLMs, see Nelder and Wedderburn 1972; McCullagh and Nelder 1989). However, although these models typically can capture over-dispersion rather well, they are in many applications not sufficient for modeling excess zeros. Since Mullahy (1986) and Lambert (1992) there is increased interest, both in the econometrics and statistics literature, in zero-augmented models that address this issue by a second model component capturing zero counts. Hurdle models (Mullahy 1986) combine a left-truncated count component with a right-censored hurdle component. Zero-inflation models (Lambert 1992) take a somewhat different approach: they are mixture models that combine a count component and a point mass at zero. An overview of count data models in econometrics, including hurdle and zero-inflated models, is provided in Cameron and Trivedi (1998, 2005).

In R (R Development Core Team 2008), GLMs are provided by the model fitting functions `glm()` (Chambers and Hastie 1992) in the **stats** package and `glm.nb()` in the **MASS** package

(Venables and Ripley 2002) along with associated methods for diagnostics and inference. Here, we discuss the implementation of hurdle and zero-inflated models in the functions `hurdle()` and `zeroinfl()` in the **pscl** package (Jackman 2008), available from the Comprehensive R Archive Network (CRAN) at http://CRAN.R-project.org/package=pscl. The design of both modeling functions as well as the methods operating on the associated fitted model objects follows that of the base R functionality so that the new software integrates easily into the computational toolbox for modeling count data in R.

The remainder of this paper is organized as follows: Section 2 discusses both the classical and zero-augmented count data models and their R implementations. In Section 3, all count regression models discussed are applied to a microeconomic cross-section data set on the demand for medical care. The summary in Section 4 concludes the main part of the paper; further technical details are presented in the appendix.

2. Models and software

In this section, we briefly outline the theory and its implementation in R (R Development Core Team 2008) for some basic count data regression models as well as their zero-augmented extensions (see Table 1 for an overview). The classical Poisson, geometric and negative binomial models are described in a generalized linear model (GLM) framework; they are implemented in R by the `glm()` function (Chambers and Hastie 1992) in the **stats** package and the `glm.nb()` function in the **MASS** package (Venables and Ripley 2002). The hurdle and zero-inflated extensions of these models are provided by the functions `hurdle()` and `zeroinfl()` in package **pscl** (Jackman 2008). The original implementation of Jackman (2008) was improved by Kleiber and Zeileis (2008) for **pscl** to make the fitting functions and the fitted

Type	Distribution	Method	Description
GLM	Poisson	ML	Poisson regression: classical GLM, estimated by maximum likelihood (ML)
		quasi	"quasi-Poisson regression": same mean function, estimated by quasi-ML (QML) or equivalently generalized estimating equations (GEE), inference adjustment via estimated dispersion parameter
		adjusted	"adjusted Poisson regression": same mean function, estimated by QML/GEE, inference adjustment via sandwich covariances
	NB	ML	NB regression: extended GLM, estimated by ML including additional shape parameter
zero-augmented	Poisson	ML	zero-inflated Poisson (ZIP), hurdle Poisson
	NB	ML	zero-inflated NB (ZINB), hurdle NB

Table 1: Overview of discussed count regression models. All GLMs use the same log-linear mean function ($\log(\mu) = x^\top\beta$) but make different assumptions about the remaining likelihood. The zero-augmented models extend the mean function by modifying (typically, increasing) the likelihood of zero counts.

model objects more similar to their `glm()` and `glm.nb()` counterparts. The most important features of the new `hurdle()` and `zeroinfl()` functions are discussed below while some technical aspects are deferred to the appendix.

An alternative implementation of zero-inflated count models is available in the currently orphaned package **zicounts** (Mwalili 2007). Another extension of zero-inflated Poisson models is available in package **ZIGP** (Erhardt 2008) which allows dispersion—in addition to mean and zero-inflation level—to depend on regressors. However, the interfaces of both packages are less standard with fewer (or no) standard methods provided. Therefore, re-using generic inference tools is more cumbersome and hence these packages are not discussed here.

Two packages that embed zero-inflated models into more general implementations of GLMs and GAMs (generalized additive models) are **gamlss** (Stasinopoulos and Rigby 2007) and **VGAM** (Yee 2008). The latter also provides hurdle models (under the name zero-altered models). Both implementations allow specification of only one set of regressors.

In addition to zero-augmented models, there are many further extensions to the classical Poisson model which are not discussed here. Some important model classes include finite mixture models—implemented in R in package **flexmix** (Leisch 2004)—and generalized estimating equations (GEE)—provided in R by package **geepack** (Halekoh, Højsgaard, and Yan 2006)—and mixed-effects models—available in R in packages **lme4** and **nlme** (see Pinheiro and Bates 2000). Further information about the models and alternative R implementations can be found in the respective references.

2.1. Generalized linear models

Model frame

The basic count data regression models can be represented and understood using the GLM framework that emerged in the statistical literature in the early 1970s (Nelder and Wedderburn 1972). In the following, we briefly sketch some important aspects relating to the unifying conceptual properties and their implementation in R—for a detailed theoretical account of GLMs see McCullagh and Nelder (1989).

GLMs describe the dependence of a scalar variable y_i $(i = 1, \ldots, n)$ on a vector of regressors x_i. The conditional distribution of $y_i|x_i$ is a linear exponential family with probability density function

$$f(y; \lambda, \phi) \quad = \quad \exp\left(\frac{y \cdot \lambda - b(\lambda)}{\phi} + c(y, \phi)\right), \tag{1}$$

where λ is the canonical parameter that depends on the regressors via a linear predictor and ϕ is a dispersion parameter that is often known. The functions $b(\cdot)$ and $c(\cdot)$ are known and determine which member of the family is used, e.g., the normal, binomial or Poisson distribution. Conditional mean and variance of y_i are given by $\mathsf{E}[y_i \,|\, x_i] = \mu_i = b'(\lambda_i)$ and $\mathsf{VAR}[y_i \,|\, x_i] = \phi \cdot b''(\lambda_i)$. Thus, up to a scale or dispersion parameter ϕ, the distribution of y_i is determined by its mean. Its variance is proportional to $V(\mu) = b''(\lambda(\mu))$, also called variance function.

The dependence of the conditional mean $\mathsf{E}[y_i \,|\, x_i] = \mu_i$ on the regressors x_i is specified via

$$g(\mu_i) \quad = \quad x_i^\top \beta, \tag{2}$$

where $g(\cdot)$ is a known link function and β is the vector of regression coefficients which are typically estimated by maximum likelihood (ML) using the iterative weighted least squares (IWLS) algorithm.

Instead of viewing GLMs as models for the full likelihood (as determined by Equation 1), they can also be regarded as regression models for the mean only (as specified in Equation 2) where the estimating functions used for fitting the model are derived from a particular family. As illustrated in the remainder of this section, the estimating function point of view is particularly useful for relaxing the assumptions imposed by the Poisson likelihood.

R provides a very flexible implementation of the general GLM framework in the function `glm()` (Chambers and Hastie 1992) contained in the **stats** package. Its most important arguments are

```
glm(formula, data, subset, na.action, weights, offset,
  family = gaussian, start = NULL, control = glm.control(...),
  model = TRUE, y = TRUE, x = FALSE, ...)
```

where `formula` plus `data` is the now standard way of specifying regression relationships in R/S introduced in Chambers and Hastie (1992). The remaining arguments in the first line (`subset`, `na.action`, `weights`, and `offset`) are also standard for setting up formula-based regression models in R/S. The arguments in the second line control aspects specific to GLMs while the arguments in the last line specify which components are returned in the fitted model object (of class "`glm`" which inherits from "`lm`"). By default the model frame (`model`) and the vector $(y_1, \ldots, y_n)^\top$ (`y`) but not the model matrix (`x`, containing $x_1, \ldots, x_n$ combined row-wise) are included. The `family` argument specifies the link $g(\mu)$ and variance function $V(\mu)$ of the model, `start` can be used to set starting values for β, and `control` contains control parameters for the IWLS algorithm. For further arguments to `glm()` (including alternative specifications of starting values) see `?glm`. The high-level `glm()` interface relies on the function `glm.fit()` which carries out the actual model fitting (without taking a formula-based input or returning classed output).

For "`glm`" objects, a set of standard methods (including `print()`, `predict()`, `logLik()` and many others) are provided. Inference can easily be performed using the `summary()` method for assessing the regression coefficients via partial Wald tests or the `anova()` method for comparing nested models via an analysis of deviance. These inference functions are complemented by further generic inference functions in contributed packages: e.g., **lmtest** (Zeileis and Hothorn 2002) provides a `coeftest()` function that also computes partial Wald tests but allows for specification of alternative (robust) standard errors. Similarly, `waldtest()` from **lmtest** and `linearHypothesis()` from **car** (Fox 2002) assess nested models via Wald tests (using different specifications for the nested models). Finally, `lrtest()` from **lmtest** compares nested models via likelihood ratio (LR) tests based on an interface similar to `waldtest()` and `anova()`.

Poisson model

The simplest distribution used for modeling count data is the Poisson distribution with probability density function

$$f(y; \mu) \quad = \quad \frac{\exp(-\mu) \cdot \mu^y}{y!}, \tag{3}$$

which is of type (1) and thus Poisson regression is a special case of the GLM framework. The canonical link is $g(\mu) = \log(\mu)$ resulting in a log-linear relationship between mean and linear predictor. The variance in the Poisson model is identical to the mean, thus the dispersion is fixed at $\phi = 1$ and the variance function is $V(\mu) = \mu$.

In R, this can easily be specified in the `glm()` call just by setting `family = poisson` (where the default log link could also be changed in the `poisson()` call).

In practice, the Poisson model is often useful for describing the mean μ_i but underestimates the variance in the data, rendering all model-based tests liberal. One way of dealing with this is to use the same estimating functions for the mean, but to base inference on the more robust sandwich covariance matrix estimator. In R, this estimator is provided by the `sandwich()` function in the **sandwich** package (Zeileis 2004, 2006).

Quasi-Poisson model

Another way of dealing with over-dispersion is to use the mean regression function and the variance function from the Poisson GLM but to leave the dispersion parameter ϕ unrestricted. Thus, ϕ is not assumed to be fixed at 1 but is estimated from the data. This strategy leads to the same coefficient estimates as the standard Poisson model but inference is adjusted for over-dispersion. Consequently, both models (quasi-Poisson and sandwich-adjusted Poisson) adopt the estimating function view of the Poisson model and do *not* correspond to models with fully specified likelihoods.

In R, the quasi-Poisson model with estimated dispersion parameter can also be fitted with the `glm()` function, simply setting `family = quasipoisson`.

Negative binomial models

A third way of modeling over-dispersed count data is to assume a negative binomial (NB) distribution for $y_i|x_i$ which can arise as a gamma mixture of Poisson distributions. One parameterization of its probability density function is

$$f(y; \mu, \theta) \quad = \quad \frac{\Gamma(y + \theta)}{\Gamma(\theta) \cdot y!} \cdot \frac{\mu^y \cdot \theta^\theta}{(\mu + \theta)^{y+\theta}}, \tag{4}$$

with mean μ and shape parameter θ; $\Gamma(\cdot)$ is the gamma function. For every fixed θ, this is of type (1) and thus is another special case of the GLM framework. It also has $\phi = 1$ but with variance function $V(\mu) = \mu + \frac{\mu^2}{\theta}$.

Package **MASS** (Venables and Ripley 2002) provides the family function `negative.binomial()` that can directly be plugged into `glm()` provided the argument `theta` is specified. One application would be the geometric model, the special case where $\theta = 1$, which can consequently be fitted in R by setting `family = negative.binomial(theta = 1)` in the `glm()` call.

If θ is not known but to be estimated from the data, the negative binomial model is not a special case of the general GLM—however, an ML fit can easily be computed re-using GLM methodology by iterating estimation of β given θ and vice versa. This leads to ML estimates for both β and θ which can be computed using the function `glm.nb()` from the package **MASS**. It returns a model of class "`negbin`" inheriting from "`glm`" for which appropriate methods to the generic functions described above are again available.

2.2. Hurdle models

In addition to over-dispersion, many empirical count data sets exhibit more zero observations than would be allowed for by the Poisson model. One model class capable of capturing both properties is the hurdle model, originally proposed by Mullahy (1986) in the econometrics literature (see Cameron and Trivedi 1998, 2005, for an overview). They are two-component models: A truncated count component, such as Poisson, geometric or negative binomial, is employed for positive counts, and a hurdle component models zero vs. larger counts. For the latter, either a binomial model or a censored count distribution can be employed.

More formally, the hurdle model combines a count data model $f_{\text{count}}(y; x, \beta)$ (that is left-truncated at $y = 1$) and a zero hurdle model $f_{\text{zero}}(y; z, \gamma)$ (right-censored at $y = 1$):

$$f_{\text{hurdle}}(y; x, z, \beta, \gamma) = \begin{cases} f_{\text{zero}}(0; z, \gamma) & \text{if } y = 0, \\ (1 - f_{\text{zero}}(0; z, \gamma)) \cdot f_{\text{count}}(y; x, \beta)/(1 - f_{\text{count}}(0; x, \beta)) & \text{if } y > 0 \end{cases} \tag{5}$$

The model parameters β, γ, and potentially one or two additional dispersion parameters θ (if f_{count} or f_{zero} or both are negative binomial densities) are estimated by ML, where the specification of the likelihood has the advantage that the count and the hurdle component can be maximized separately. The corresponding mean regression relationship is given by

$$\log(\mu_i) \quad = \quad x_i^\top \beta + \log(1 - f_{\text{zero}}(0; z_i, \gamma)) - \log(1 - f_{\text{count}}(0; x_i, \beta)), \tag{6}$$

again using the canonical log link. For interpreting the zero model as a hurdle, a binomial GLM is probably the most intuitive specification[1]. Another useful interpretation arises if the same regressors $x_i = z_i$ are used in the same count model in both components $f_{\text{count}} = f_{\text{zero}}$: A test of the hypothesis $\beta = \gamma$ then tests whether the hurdle is needed or not.

In R, hurdle count data models can be fitted with the `hurdle()` function from the **pscl** package (Jackman 2008). Both its fitting function and the returned model objects of class "`hurdle`" are modelled after the corresponding GLM functionality in R. The arguments of `hurdle()` are given by

```
hurdle(formula, data, subset, na.action, weights, offset,
  dist = "poisson", zero.dist = "binomial", link = "logit",
  control = hurdle.control(...),
  model = TRUE, y = TRUE, x = FALSE, ...)
```

where the first line contains the standard model-frame specifications, the second and third lines have the arguments specific to hurdle models and the arguments in the last line control some components of the return value.

If a `formula` of type `y ~ x1 + x2` is supplied, it not only describes the count regression relationship of y_i and x_i but also implies that the same set of regressors is used for the zero hurdle component $z_i = x_i$. This is could be made more explicit by equivalently writing the formula as `y ~ x1 + x2 | x1 + x2`. Of course, a different set of regressors could be specified for the zero hurdle component, e.g., `y ~ x1 + x2 | z1 + z2 + z3`, giving the count data model `y ~ x1 + x2` conditional on (`|`) the zero hurdle model `y ~ z1 + z2 + z3`.

[1] Note that binomial logit and censored geometric models as the hurdle part both lead to the same likelihood function and thus to the same coefficient estimates (Mullahy 1986).

The model likelihood can be specified by the `dist`, `zero.dist` and `link` arguments. The count data distribution `dist` is `"poisson"` by default (it can also be set to `"negbin"` or `"geometric"`), for which the canonical log link is always used. The distribution for the zero hurdle model can be specified via `zero.dist`. The default is a binomial model with `link` (defaulting to `"logit"`, but all link functions of the `binomial()` family are also supported), alternatively a right-censored count distribution (Poisson, negative binomial or geometric, all with log link) could be specified.

ML estimation of all parameters employing analytical gradients is carried out using R's `optim()` with control options set in `hurdle.control()`. Starting values can be user-supplied, otherwise they are estimated by `glm.fit()` (the default). The covariance matrix estimate is derived numerically using the Hessian matrix returned by `optim()`. See Appendix A for further technical details.

The returned fitted-model object of class "`hurdle`" is a list similar to "`glm`" objects. Some of its elements—such as `coefficients` or `terms`—are lists with a zero and count component, respectively. For details see Appendix A.

A set of standard extractor functions for fitted model objects is available for objects of class "`hurdle`", including the usual `summary()` method that provides partial Wald tests for all coefficients. No `anova()` method is provided, but the general `coeftest()`, `waldtest()` from **lmtest**, and `linearHypothesis()` from **car** can be used for Wald tests and `lrtest()` from **lmtest** for LR tests of nested models. The function `hurdletest()` is a convenience interface to `linearHypothesis()` for testing for the presence of a hurdle (which is only applicable if the same regressors and the same count distribution are used in both components).

2.3. Zero-inflated models

Zero-inflated models (Mullahy 1986; Lambert 1992) are another model class capable of dealing with excess zero counts (see Cameron and Trivedi 1998, 2005, for an overview). They are two-component mixture models combining a point mass at zero with a count distribution such as Poisson, geometric or negative binomial. Thus, there are two sources of zeros: zeros may come from both the point mass and from the count component. For modeling the unobserved state (zero vs. count), a binary model is used: in the simplest case only with an intercept but potentially containing regressors.

Formally, the zero-inflated density is a mixture of a point mass at zero $I_{\{0\}}(y)$ and a count distribution $f_{\text{count}}(y; x, \beta)$. The probability of observing a zero count is inflated with probability $\pi = f_{\text{zero}}(0; z, \gamma)$:

$$f_{\text{zeroinfl}}(y; x, z, \beta, \gamma) \quad = \quad f_{\text{zero}}(0; z, \gamma) \cdot I_{\{0\}}(y) \; + \; (1 - f_{\text{zero}}(0; z, \gamma)) \cdot f_{\text{count}}(y; x, \beta), \tag{7}$$

where $I(\cdot)$ is the indicator function and the unobserved probability π of belonging to the point mass component is modelled by a binomial GLM $\pi = g^{-1}(z^\top \gamma)$. The corresponding regression equation for the mean is

$$\mu_i \quad = \quad \pi_i \cdot 0 \; + \; (1 - \pi_i) \cdot \exp(x_i^\top \beta), \tag{8}$$

using the canonical log link. The vector of regressors in the zero-inflation model z_i and the regressors in the count component x_i need not to be distinct—in the simplest case, $z_i = 1$ is just an intercept. The default link function $g(\pi)$ in binomial GLMs is the logit link, but other

links such as the probit are also available. The full set of parameters of β, γ, and potentially the dispersion parameter θ (if a negative binomial count model is used) can be estimated by ML. Inference is typically performed for β and γ, while θ is treated as a nuisance parameter even if a negative binomial model is used.

In R, zero-inflated count data models can be fitted with the `zeroinfl()` function from the **pscl** package. Both the fitting function interface and the returned model objects of class "`zeroinfl`" are almost identical to the corresponding `hurdle()` functionality and again modelled after the corresponding GLM functionality in R. The arguments of `zeroinfl()` are given by

```
zeroinfl(formula, data, subset, na.action, weights, offset,
  dist = "poisson", link = "logit", control = zeroinfl.control(...),
  model = TRUE, y = TRUE, x = FALSE, ...)
```

where all arguments have almost the same meaning as for `hurdle()`. The main difference is that there is no `zero.dist` argument: a binomial model is always used for distribution in the zero-inflation component.

Again, ML estimates of all parameters are obtained from `optim()`, with control options set in `zeroinfl.control()` and employing analytical gradients. Starting values can be user-supplied, estimated by the expectation maximization (EM) algorithm, or by `glm.fit()` (the default). The covariance matrix estimate is derived numerically using the Hessian matrix returned by `optim()`. Using EM estimation for deriving starting values is typically slower but can be numerically more stable. It already maximizes the likelihood, but a single `optim()` iteration is used for determining the covariance matrix estimate. See Appendix B for further technical details.

The returned fitted model object is of class "`zeroinfl`" whose structure is virtually identical to that of "`hurdle`" models. As above, a set of standard extractor functions for fitted model objects is available for objects of class "`zeroinfl`", including the usual `summary()` method that provides partial Wald tests for all coefficients. Again, no `anova()` method is provided, but the general functions `coeftest()` and `waldtest()` from **lmtest**, as well as `linearHypothesis()` from **car** can be used for Wald tests, and `lrtest()` from **lmtest** for LR tests of nested models.

3. Application and illustrations

In the following, we illustrate all models described above by applying them to a cross-sectional data set from health economics. Before the parametric models are fitted, a basic exploratory analysis of the data set is carried out that addresses some problems typically encountered when visualizing count data. At the end of the section, all fitted models are compared highlighting that the modelled mean function is similar but the fitted likelihood is different and thus, the models differ with respect to explaining over-dispersion and/or the number of zero counts.

3.1. Demand for medical care by the elderly

Deb and Trivedi (1997) analyze data on 4406 individuals, aged 66 and over, who are covered by Medicare, a public insurance program. Originally obtained from the US National Medical

Expenditure Survey (NMES) for 1987/88, the data are available from the data archive of the *Journal of Applied Econometrics* at http://www.econ.queensu.ca/jae/1997-v12.3/deb-trivedi/. It was prepared for an R package accompanying Kleiber and Zeileis (2008) and is also available as `DebTrivedi.rda` in the *Journal of Statistical Software* together with Zeileis (2006). The objective is to model the demand for medical care—as captured by the number of physician/non-physician office and hospital outpatient visits—by the covariates available for the patients. Here, we adopt the number of physician office visits `ofp` as the dependent variable and use the health status variables `hosp` (number of hospital stays), `health` (self-perceived health status), `numchron` (number of chronic conditions), as well as the socio-economic variables `gender`, `school` (number of years of education), and `privins` (private insurance indicator) as regressors. For convenience, we select the variables used from the full data set:

```
R> dt <- DebTrivedi[, c(1, 6:8, 13, 15, 18)]
```

To obtain a first overview of the dependent variable, we employ a histogram of the observed count frequencies. In R various tools could be used, e.g., `hist(dt$ofp, breaks = 0:90 - 0.5)` for a histogram with rectangles or

```
R> plot(table(dt$ofp))
```

(see Figure 1) for a histogram with lines which brings out the extremely large counts somewhat better. The histogram illustrates that the marginal distribution exhibits both substantial variation and a rather large number of zeros.

A natural second step in the exploratory analysis is to look at pairwise bivariate displays of the dependent variable against each of the regressors bringing out the partial relationships. In R, such bivariate displays can easily be generated with the `plot()` method for formulas, e.g., via `plot(y ~ x)`. This chooses different types of displays depending on the combination of quantitative and qualitative variables as dependent or regressor variable, respectively. However, count variables are treated as all numerical variables and therefore the command

```
R> plot(ofp ~ numchron, data = dt)
```

produces a simple scatterplot as shown in the left panel of Figure 2. This is clearly not useful as both variables are count variables producing numerous ties in the bivariate distribution and thus obscuring a large number of points in the display. To overcome the problem, it is useful to group the number of chronic conditions into a factor with levels '0', '1', '2', and '3 or more' and produce a boxplot instead of a scatterplot. Furthermore, the picture is much clearer if the dependent variable is log-transformed (just as all count regression models discussed above also use a log link by default). As there are zero counts as well, we use a convenience function `clog()` providing a continuity-corrected logarithm.

```
R> clog <- function(x) log(x + 0.5)
```

For transforming a count variable to a factor (for visualization purposes only), we define another convenience function `cfac()`

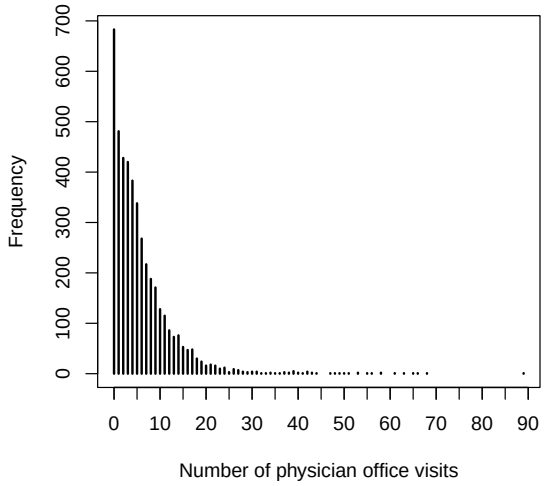


Figure 1: Frequency distribution for number of physician office visits.

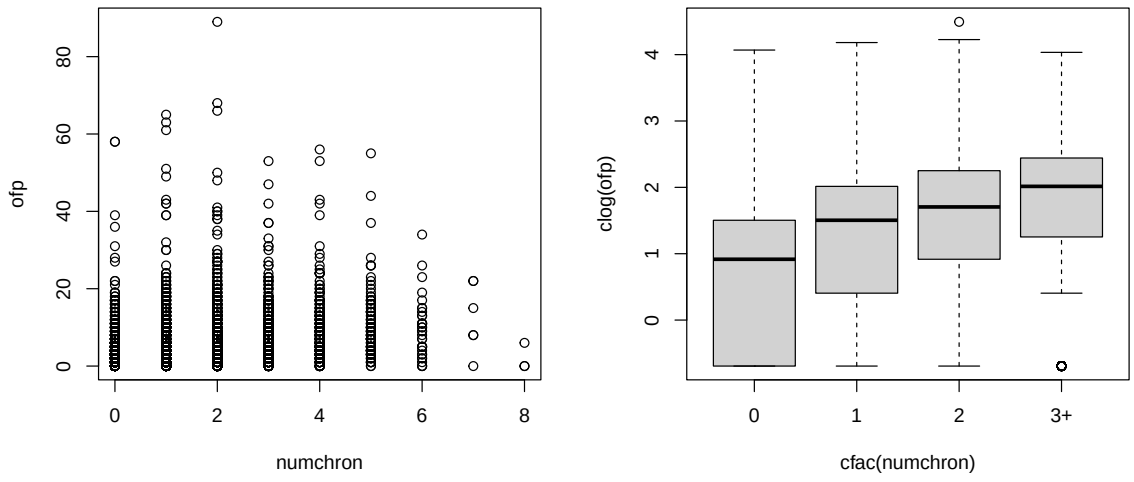


Figure 2: Bivariate explorative displays for number of physician office visits plotted against number of chronic conditions.

```
R> cfac <- function(x, breaks = NULL) {
+   if(is.null(breaks)) breaks <- unique(quantile(x, 0:10/10))
+   x <- cut(x, breaks, include.lowest = TRUE, right = FALSE)
+   levels(x) <- paste(breaks[-length(breaks)], ifelse(diff(breaks) > 1,
+     c(paste("-", breaks[-c(1, length(breaks))] - 1, sep = ""), "+"), ""),
+     sep = "")
+   return(x)
+ }
```

which by default tries to take an educated guess how to choose the breaks between the categories. Clearly, the resulting exploratory display of the transformed variables produced by

```
R> plot(clog(ofp) ~ cfac(numchron), data = dt)
```

(shown in the right panel of Figure 2) brings out much better how the number of doctor visits increases with the number of chronic conditions.

Analogous displays for the number of physician office visits against all regressors can be produced via

```
R> plot(clog(ofp) ~ health, data = dt, varwidth = TRUE)
R> plot(clog(ofp) ~ cfac(numchron), data = dt)
R> plot(clog(ofp) ~ privins, data = dt, varwidth = TRUE)
R> plot(clog(ofp) ~ cfac(hosp, c(0:2, 8)), data = dt)
R> plot(clog(ofp) ~ gender, data = dt, varwidth = TRUE)
R> plot(cfac(ofp, c(0:2, 4, 6, 10, 100)) ~ school, data = dt, breaks = 9)
```

and are shown (with slightly enhanced labeling) in Figure 3. The last plot uses a different type of display. Here, the dependent count variable is not log-transformed but grouped into a factor and then a spinogram is produced. This also groups the regressor (as in a histogram) and then produces a highlighted mosaic plot. All displays show that the number of doctor visits increases or decreases with the regressors as expected: `ofp` decreases with the general health status but increases with the number of chronic conditions or hospital stays. The median number of visits is also slightly higher for patients with a private insurance and higher level of education. It is slightly lower for male compared to female patients. The overall impression from all displays is that the changes in the mean can only explain a modest amount of variation in the data.

3.2. Poisson regression

As a first attempt to capture the relationship between the number of physician office visits and all regressors—described in R by the formula `ofp ~ .`—in a parametric regression model, we fit the basic Poisson regression model

```
R> fm_pois <- glm(ofp ~ ., data = dt, family = poisson)
```

and obtain the coefficient estimates along with associated partial Wald tests

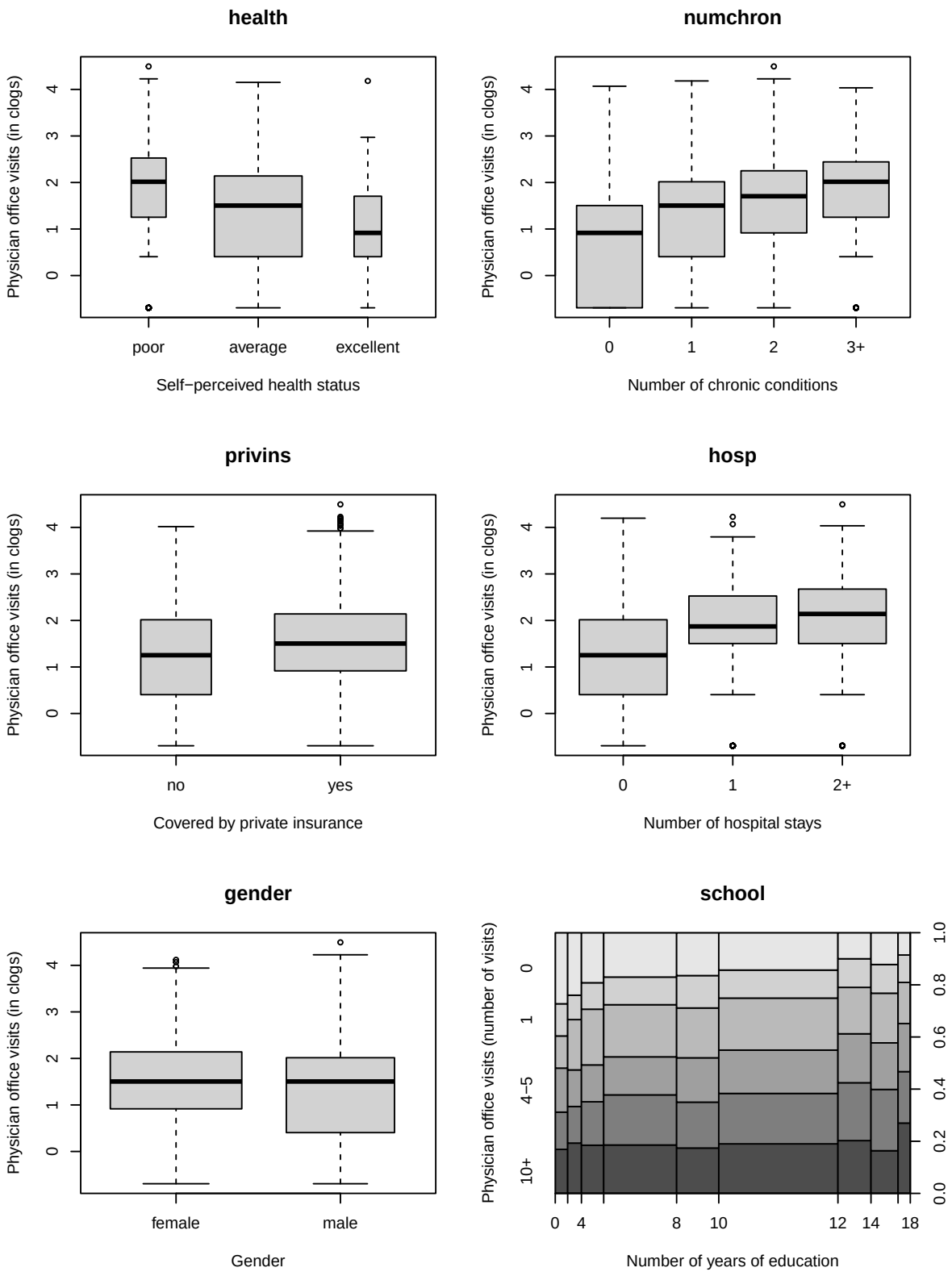


Figure 3: Number of physician office visits plotted against regressors used.

```
R> summary(fm_pois)

Call:
glm(formula = ofp ~ ., family = poisson, data = dt)

Coefficients:
                 Estimate Std. Error z value Pr(>|z|)
(Intercept)      1.028874   0.023785  43.258   <2e-16 ***
hosp             0.164797   0.005997  27.478   <2e-16 ***
healthpoor       0.248307   0.017845  13.915   <2e-16 ***
healthexcellent -0.361993   0.030304 -11.945   <2e-16 ***
numchron         0.146639   0.004580  32.020   <2e-16 ***
gendermale      -0.112320   0.012945  -8.677   <2e-16 ***
school           0.026143   0.001843  14.182   <2e-16 ***
privinsyes       0.201687   0.016860  11.963   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 26943  on 4405  degrees of freedom
Residual deviance: 23168  on 4398  degrees of freedom
AIC: 35959

Number of Fisher Scoring iterations: 5
```

All coefficient estimates confirm the results from the exploratory analysis in Figure 3. All coefficients are highly significant with the health variables leading to somewhat larger Wald statistics compared to the socio-economic variables. However, the Wald test results might be too optimistic due to a misspecification of the likelihood. As the exploratory analysis suggested that over-dispersion is present in this data set, we re-compute the Wald tests using sandwich standard errors via

```
R> coeftest(fm_pois, vcov = sandwich)

z test of coefficients:

                 Estimate Std. Error z value  Pr(>|z|)
(Intercept)      1.028874   0.064530 15.9442 < 2.2e-16 ***
hosp             0.164797   0.021945  7.5095 5.935e-14 ***
healthpoor       0.248307   0.054022  4.5964 4.298e-06 ***
healthexcellent -0.361993   0.077449 -4.6740 2.954e-06 ***
numchron         0.146639   0.012908 11.3605 < 2.2e-16 ***
gendermale      -0.112320   0.035343 -3.1780  0.001483 **
school           0.026143   0.005084  5.1422 2.715e-07 ***
privinsyes       0.201687   0.043128  4.6765 2.919e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

All regressors are still significant but the standard errors seem to be more appropriate. This will also be confirmed by the following models that deal with over-dispersion (and excess zeros) in a more formal way.

3.3. Quasi-Poisson regression

The quasi-Poisson model

```
R> fm_qpois <- glm(ofp ~ ., data = dt, family = quasipoisson)
```

leads to an estimated dispersion of $\hat{\phi} = 6.706$ which is clearly larger than 1 confirming that over-dispersion is present in the data.[2] The resulting partial Wald tests of the coefficients are rather similar to the results obtained from the Poisson regression with sandwich standard errors, leading to the same conclusions. As before, they can be obtained via

```
R> summary(fm_qpois)
```

The output is suppressed here and is presented in tabular form in Table 2.

3.4. Negative binomial regression

A more formal way to accommodate over-dispersion in a count data regression model is to use a negative binomial model, as in

```
R> fm_nbin <- MASS::glm.nb(ofp ~ ., data = dt)
R> summary(fm_nbin)
```

As shown in Table 2, both regression coefficients and standard errors are rather similar to the quasi-Poisson and the sandwich-adjusted Poisson results above. Thus, in terms of predicted means all three models give very similar results; the associated partial Wald tests also lead to the same conclusions.

One advantage of the negative binomial model is that it is associated with a formal likelihood so that information criteria are readily available. Furthermore, the expected number of zeros can be computed from the fitted densities via $\sum_i f(0, \hat{\mu}_i, \hat{\theta})$.

3.5. Hurdle regression

The exploratory analysis conveyed the impression that there might be more zero observations than explained by the basic count data distributions, hence a negative binomial hurdle model is fitted via

```
R> fm_hurdle0 <- hurdle(ofp ~ ., data = dt, dist = "negbin")
```

This uses the same type of count data model as in the preceding section but it is now truncated for `ofp < 1` and has an additional hurdle component modeling zero vs. count observations. By default, the hurdle component is a binomial GLM with logit link which contains all regressors used in the count model. The associated coefficient estimates and partial Wald tests for both model components are displayed via

[2] Alternatively, over-dispersion can be confirmed by comparison of the log-likelihoods of the Poisson and negative binomial model.

```
R> summary(fm_hurdle0)

Call:
hurdle(formula = ofp ~ ., data = dt, dist = "negbin")

Pearson residuals:
    Min      1Q  Median      3Q     Max
-1.1718 -0.7080 -0.2737  0.3196 18.0092

Count model coefficients (truncated negbin with log link):
                 Estimate Std. Error z value Pr(>|z|)
(Intercept)      1.197699   0.058973  20.309  < 2e-16 ***
hosp             0.211898   0.021396   9.904  < 2e-16 ***
healthpoor       0.315958   0.048056   6.575 4.87e-11 ***
healthexcellent -0.331861   0.066093  -5.021 5.14e-07 ***
numchron         0.126421   0.012452  10.152  < 2e-16 ***
gendermale      -0.068317   0.032416  -2.108   0.0351 *
school           0.020693   0.004535   4.563 5.04e-06 ***
privinsyes       0.100172   0.042619   2.350   0.0188 *
Log(theta)       0.333255   0.042754   7.795 6.46e-15 ***
Zero hurdle model coefficients (binomial with logit link):
                 Estimate Std. Error z value Pr(>|z|)
(Intercept)      0.043147   0.139852   0.309 0.757688
hosp             0.312449   0.091437   3.417 0.000633 ***
healthpoor      -0.008716   0.161024  -0.054 0.956833
healthexcellent -0.289570   0.142682  -2.029 0.042409 *
numchron         0.535213   0.045378  11.794  < 2e-16 ***
gendermale      -0.415658   0.087608  -4.745 2.09e-06 ***
school           0.058541   0.011989   4.883 1.05e-06 ***
privinsyes       0.747120   0.100880   7.406 1.30e-13 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Theta: count = 1.3955
Number of iterations in BFGS optimization: 16
Log-likelihood: -1.209e+04 on 17 Df
```

The coefficients in the count component resemble those from the previous models, but the increase in the log-likelihood (see also Table 2) conveys that the model has improved by including the hurdle component. However, it might be possible to omit the `health` variable from the hurdle model. To test this hypothesis, the reduced model is fitted via

```
R> fm_hurdle <- hurdle(ofp ~ . | hosp + numchron + privins + school + gender,
+   data = dt, dist = "negbin")
```

and can then be compared to the full model in a Wald test

```
R> waldtest(fm_hurdle0, fm_hurdle)
```

```
Wald test

Model 1: ofp ~ .
Model 2: ofp ~ . | hosp + numchron + privins + school + gender
  Res.Df Df  Chisq Pr(>Chisq)
1   4389
2   4391 -2 4.1213     0.1274
```

or an LR test

```
R> lrtest(fm_hurdle0, fm_hurdle)
```

which leads to virtually identical results.

3.6. Zero-inflated regression

A different way of augmenting the negative binomial count model `fm_nbin` with additional probability weight for zero counts is a zero-inflated negative binomial (ZINB) regression. The default model is fitted via

```
R> fm_zinb0 <- zeroinfl(ofp ~ ., data = dt, dist = "negbin")
```

As for the hurdle model above, all regressors from the count model are also used in the zero-inflation model. Again, we can modify the regressors in the zero-inflation part, e.g., by fitting a second model

```
R> fm_zinb <- zeroinfl(ofp ~ . | hosp + numchron + privins + school + gender,
+   data = dt, dist = "negbin")
```

that has the same variables in the zero-inflation part as the hurdle component in `fm_hurdle`. By omitting the `health` variable, the fit does not change significantly which can again be brought out by a Wald test

```
R> waldtest(fm_zinb0, fm_zinb)
```

```
Wald test

Model 1: ofp ~ .
Model 2: ofp ~ . | hosp + numchron + privins + school + gender
  Res.Df Df  Chisq Pr(>Chisq)
1   4389
2   4391 -2 0.1584     0.9238
```

or an LR test `lrtest(fm_zinb0, fm_zinb)` that produces virtually identical results. The chosen fitted model can again be inspected via

```
R> summary(fm_zinb)
```

Type	GLM				zero-augmented	
Distribution		Poisson		NB	Hurdle-NB	ZINB
Method	ML	adjusted	quasi	ML	ML	ML
Object	`fm_pois`	`fm_pois`	`fm_qpois`	`fm_nbin`	`fm_hurdle`	`fm_zinb`
`(Intercept)`	1.029	1.029	1.029	0.929	1.198	1.194
	(0.024)	(0.065)	(0.062)	(0.055)	(0.059)	(0.057)
`hosp`	0.165	0.165	0.165	0.218	0.212	0.201
	(0.006)	(0.022)	(0.016)	(0.020)	(0.021)	(0.020)
`healthpoor`	0.248	0.248	0.248	0.305	0.316	0.285
	(0.018)	(0.054)	(0.046)	(0.049)	(0.048)	(0.045)
`healthexcellent`	−0.362	−0.362	−0.362	−0.342	−0.332	−0.319
	(0.030)	(0.077)	(0.078)	(0.061)	(0.066)	(0.060)
`numchron`	0.147	0.147	0.147	0.175	0.126	0.129
	(0.005)	(0.013)	(0.012)	(0.012)	(0.012)	(0.012)
`gendermale`	−0.112	−0.112	−0.112	−0.126	−0.068	−0.080
	(0.013)	(0.035)	(0.034)	(0.031)	(0.032)	(0.031)
`school`	0.026	0.026	0.026	0.027	0.021	0.021
	(0.002)	(0.005)	(0.005)	(0.004)	(0.005)	(0.004)
`privinsyes`	0.202	0.202	0.202	0.224	0.100	0.126
	(0.017)	(0.043)	(0.044)	(0.039)	(0.043)	(0.042)
`(Intercept)`					0.016	−0.047
					(0.138)	(0.269)
`hosp`					0.318	−0.800
					(0.091)	(0.421)
`numchron`					0.548	−1.248
					(0.044)	(0.178)
`privinsyes`					0.746	−1.176
					(0.100)	(0.220)
`school`					0.057	−0.084
					(0.012)	(0.026)
`gendermale`					−0.419	0.648
					(0.088)	(0.200)
no. parameters	8	8	9	9	15	15
$\log L$	−17971.6			−12170.6	−12090.1	−12090.7
AIC	35959.2			24359.1	24210.1	24211.4
BIC	36010.4			24416.6	24306.0	24307.3
$\sum_i \hat{f}_i(0)$	47			608	683	709

Table 2: Summary of fitted count regression models for NMES data: coefficient estimates from count model, zero-inflation model (both with standard errors in parentheses), number of estimated parameters, maximized log-likelihood, AIC, BIC and expected number of zeros (sum of fitted densities evaluated at zero). The observed number of zeros is 683 in 4406 observations.

See Table 2 for a more concise summary.

3.7. Comparison

Having fitted several count data regression models to the demand for medical care in the NMES data, it is, of course, of interest to understand what these models have in common and what their differences are. In this section, we show how to compute the components of Table 2 and provide some further comments and interpretations.

As a first comparison, it is of natural interest to inspect the estimated regression coefficients in the count data model

```
R> fm <- list("ML-Pois" = fm_pois, "Quasi-Pois" = fm_qpois, "NB" = fm_nbin,
+   "Hurdle-NB" = fm_hurdle, "ZINB" = fm_zinb)
R> sapply(fm, function(x) coef(x)[1:8])
```

The result (see Table 2) shows that there are some small differences, especially between the GLMs and the zero-augmented models. However, the zero-augmented models have to be interpreted slightly differently: While the GLMs all have the same mean function (2), the zero-augmentation also enters the mean function, see (8) and (6). Nevertheless, the overall impression is that the estimated mean functions are rather similar. Moreover, the associated estimated standard errors are very similar as well (see Table 2):

```
R> cbind("ML-Pois" = sqrt(diag(vcov(fm_pois))),
+   "Adj-Pois" = sqrt(diag(sandwich(fm_pois))),
+   sapply(fm[-1], function(x) sqrt(diag(vcov(x)))[1:8]))
```

The only exception are the model-based standard errors for the Poisson model, when treated as a fully specified model, which is obviously not appropriate for this data set.

In summary, the models are not too different with respect to their fitted mean functions. The differences become obvious if not only the mean but the full likelihood is considered:

```
R> rbind(logLik = sapply(fm, function(x) round(logLik(x), digits = 0)),
+   Df = sapply(fm, function(x) attr(logLik(x), "df")))
```

```
       ML-Pois Quasi-Pois     NB Hurdle-NB   ZINB
logLik  -17972         NA -12171    -12090 -12091
Df            8          9      9        15     15
```

The ML Poisson model is clearly inferior to all other fits. The quasi-Poisson model and the sandwich-adjusted Poisson model are not associated with a fitted likelihood. The negative binomial already improves the fit dramatically but can in turn be improved by the hurdle and zero-inflated models which give almost identical fits. This also reflects that the over-dispersion in the data is captured better by the negative-binomial-based models than the plain Poisson model. Additionally, it is of interest how the zero counts are captured by the various models. Therefore, the observed zero counts are compared to the expected number of zero counts for the likelihood-based models:

```
R> round(c("Obs" = sum(dt$ofp < 1),
+   "ML-Pois" = sum(dpois(0, fitted(fm_pois))),
+   "NB" = sum(dnbinom(0, mu = fitted(fm_nbin), size = fm_nbin$theta)),
+   "NB-Hurdle" = sum(predict(fm_hurdle, type = "prob")[,1]),
+   "ZINB" = sum(predict(fm_zinb, type = "prob")[,1])))

      Obs   ML-Pois        NB NB-Hurdle      ZINB
      683        47       608       683       709
```

Thus, the ML Poisson model is again not appropriate whereas the negative-binomial-based models are much better in modeling the zero counts. By construction, the expected number of zero counts in the hurdle model matches the observed number.

In summary, the hurdle and zero-inflation models lead to the best results (in terms of likelihood) on this data set. Above, their mean function for the count component was already shown to be very similar, below we take a look at the fitted zero components:

```
R> t(sapply(fm[4:5], function(x) round(x$coefficients$zero, digits = 3)))

          (Intercept)   hosp numchron privinsyes school gendermale
Hurdle-NB       0.016  0.318    0.548      0.746  0.057     -0.419
ZINB           -0.047 -0.800   -1.248     -1.176 -0.084      0.648
```

This shows that the absolute values are rather different—which is not surprising as they pertain to slightly different ways of modeling zero counts—but the signs of the coefficients match, i.e., are just inversed. For the hurdle model, the zero hurdle component describes the probability of observing a positive count whereas, for the ZINB model, the zero-inflation component predicts the probability of observing a zero count from the point mass component. Overall, both models lead to the same qualitative results and very similar model fits. Perhaps the hurdle model is slightly preferable because it has the nicer interpretation: there is one process that controls whether a patient sees a physician or not, and a second process that determines how many office visits are made.

4. Summary

The model frame for basic count data models from the GLM framework as well as their implementation in the R system for statistical computing is reviewed. Starting from these basic tools, it is presented how hurdle and zero-inflated models extend the classical models and how likewise their R implementation in package **pscl** re-uses design and functionality of the corresponding R software. Hence, the new functions `hurdle()` and `zeroinfl()` are straightforward to apply for model fitting. Additionally, standard methods for diagnostics are provided and generic inference tools from other packages can easily be re-used.

Computational details

The results in this paper were obtained using R 4.4.0 with the packages **MASS** 7.3–60.2, **pscl** 1.5.9, **sandwich** 3.1–0, **car** 3.1–2, **lmtest** 0.9–40. R itself and all packages used are available from CRAN at http://CRAN.R-project.org/.

References

Cameron AC, Trivedi PK (1998). *Regression Analysis of Count Data*. Cambridge University Press, Cambridge.

Cameron AC, Trivedi PK (2005). *Microeconometrics: Methods and Applications*. Cambridge University Press, Cambridge.

Chambers JM, Hastie TJ (eds.) (1992). *Statistical Models in S*. Chapman & Hall, London.

Deb P, Trivedi PK (1997). "Demand for Medical Care by the Elderly: A Finite Mixture Approach." *Journal of Applied Econometrics*, **12**, 313–336.

Erhardt V (2008). ***ZIGP**: Zero-inflated Generalized Poisson Regression Models*. R package version 2.1, URL http://CRAN.R-project.org/package=ZIGP.

Fox J (2002). *An R and S-PLUS Companion to Applied Regression*. Sage Publications, Thousand Oaks, CA.

Halekoh U, Højsgaard S, Yan J (2006). "The R Package **geepack** for Generalized Estimating Equations." *Journal of Statistical Software*, **15**(2), 1–11. URL http://www.jstatsoft.org/v15/i02/.

Jackman S (2008). ***pscl**: Classes and Methods for R Developed in the Political Science Computational Laboratory, Stanford University*. Department of Political Science, Stanford University, Stanford, California. R package version 0.95, URL http://CRAN.R-project.org/package=pscl.

Kleiber C, Zeileis A (2008). *Applied Econometrics with R*. Springer-Verlag, New York. ISBN 978-0-387-77316-2.

Lambert D (1992). "Zero-inflated Poisson Regression, With an Application to Defects in Manufacturing." *Technometrics*, **34**, 1–14.

Leisch F (2004). "FlexMix: A General Framework for Finite Mixture Models and Latent Class Regression in R." *Journal of Statistical Software*, **11**(8), 1–18. URL http://www.jstatsoft.org/v11/i08/.

McCullagh P, Nelder JA (1989). *Generalized Linear Models*. 2nd edition. Chapman & Hall, London.

Mullahy J (1986). "Specification and Testing of Some Modified Count Data Models." *Journal of Econometrics*, **33**, 341–365.

Mwalili SM (2007). ***zicounts**: Classical and Censored Zero-inflated Count Data Models*. R package version 1.1.5 (orphaned), URL http://CRAN.R-project.org/src/contrib/Archive/zicounts/.

Nelder JA, Wedderburn RWM (1972). "Generalized Linear Models." *Journal of the Royal Statistical Society A*, **135**, 370–384.

Pinheiro JC, Bates DM (2000). *"Mixed-Effects Models in S and S-PLUS*. Springer-Verlag, New York.

R Development Core Team (2008). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. ISBN 3-900051-00-3, URL http://www.R-project.org/.

Stasinopoulos DM, Rigby RA (2007). "Generalized Additive Models for Location Scale and Shape (GAMLSS) in R." *Journal of Statistical Software*, **23**(7). URL http://www.jstatsoft.org/v23/i07/.

Venables WN, Ripley BD (2002). *Modern Applied Statistics with S*. 4th edition. Springer-Verlag, New York.

Yee TW (2008). ***VGAM**: Vector Generalized Linear and Additive Models*. R package version 0.7-7, URL http://CRAN.R-project.org/package=VGAM.

Zeileis A (2004). "Econometric Computing with HC and HAC Covariance Matrix Estimators." *Journal of Statistical Software*, **11**(10), 1–17. URL http://www.jstatsoft.org/v11/i10/.

Zeileis A (2006). "Object-oriented Computation of Sandwich Estimators." *Journal of Statistical Software*, **16**(9), 1–16. URL http://www.jstatsoft.org/v16/i09/.

Zeileis A, Hothorn T (2002). "Diagnostic Checking in Regression Relationships." *R News*, **2**(3), 7–10. URL http://CRAN.R-project.org/doc/Rnews/.

Zeileis A, Kleiber C (2008). ***AER**: Applied Econometrics with R*. R package version 0.9-0, URL http://CRAN.R-project.org/package=AER.

A. Technical details for hurdle models

The fitting of hurdle models via ML in `hurdle()` is controlled by the arguments in the `hurdle.control()` wrapper function:

```
hurdle.control(method = "BFGS", maxit = 10000, trace = FALSE,
  separate = TRUE, start = NULL, ...)
```

This modifies some default arguments passed on to the optimizer `optim()`, such as `method`, `maxit` and `trace`. The latter is also used within `hurdle()` and can be set to produce more verbose output concerning the fitting process. The argument `separate` controls whether the two components of the model are optimized separately (the default) or not. This is possible because there are no mixed sources for the zeros in the data (unlike in zero-inflation models). The argument `start` controls the choice of starting values for calling `optim()`, all remaining arguments passed through `...` are directly passed on to `optim()`.

By default, starting values are estimated by calling `glm.fit()` for both components of the model separately, once for the counts and once for zero vs. non-zero counts. If starting values are supplied, `start` needs to be set to a named list with the parameters for the `$count` and `$zero` part of the model (and potentially a `$theta` dispersion parameter if a negative binomial distribution is used).

The fitted model object of class "`hurdle`" is similar to "`glm`" objects and contains sufficient information on all aspects of the fitting process. In particular, the estimated parameters and associated covariances are included as well as the result from the `optim()` call. Furthermore, the call, formula, terms structure etc. is contained, potentially also the model frame, dependent variable and regressor matrices.

Following `glm.nb()`, the θ parameter of the negative binomial distribution is treated as a nuisance parameter. Thus, the `$coefficients` component of the fitted model object just contains estimates of β and γ while the estimate of θ and its standard deviation (on a log scale) are kept in extra list elements `$theta` and `$SE.logtheta`.

B. Technical details for zero-inflated models

Both the interface of the `zeroinfl()` function as well as its fitted model objects are virtually identical to the corresponding "`hurdle`" functionality. Hence, we only provide some additional information for those aspects that differ from those discussed above. The details of the ML optimization are again provided by a `zeroinfl.control()` wrapper:

```
zeroinfl.control(method = "BFGS", maxit = 10000, trace = FALSE,
  EM = FALSE, start = NULL, ...)
```

The only new argument here is the argument `EM` which allows for EM estimation of the starting values. Instead of calling `glm.fit()` only once for both components of the model, this process can be iterated until convergence of the parameters to the ML estimates. The optimizer is still called subsequently (for a single iteration) to obtain the Hessian matrix from which the estimated covariance matrix can be computed.

C. Methods for fitted zero-inflated and hurdle models

Users typically should not need to compute on the internal structure of "`hurdle`" or "`zeroinfl`" objects because a set of standard extractor functions is provided, an overview is given in Table 3. This includes methods to the generic functions `print()` and `summary()` which print the estimated coefficients along with further information. The `summary()` in particular supplies partial Wald tests based on the coefficients and the covariance matrix. As usual, the `summary()` method returns an object of class "`summary.hurdle`" or "`summary.zeroinfl`", respectively, containing the relevant summary statistics which can subsequently be printed using the associated `print()` method.

The methods for `coef()` and `vcov()` by default return a single vector of coefficients and their associated covariance matrix, respectively, i.e., all coefficients are concatenated. By setting their `model` argument, the estimates for a single component can be extracted. Concatenating the parameters by default and providing a matching covariance matrix estimate (that does not contain the covariances of further nuisance parameters) facilitates the application of generic inference functions such as `coeftest()`, `waldtest()`, and `linearHypothesis()`. All of these compute Wald tests for which coefficient estimates and associated covariances is essentially all information required and can therefore be queried in an object-oriented way with the `coef()` and `vcov()` methods.

Similarly, the `terms()` and `model.matrix()` extractors can be used to extract the relevant information for either component of the model. A `logLik()` method is provided, hence `AIC()` can be called to compute information criteria and `lrtest()` for conducting LR tests of nested models.

The `predict()` method computes predicted means (default) or probabilities (i.e., likelihood contributions) for observed or new data. Additionally, the means from the count and zero component, respectively, can be predicted. For the count component, this is the predicted count mean (without hurdle/inflation): $\exp(x_i^\top \beta)$. For the zero component, this is the the ratio of probabilities $(1 - f_{\text{zero}}(0; z_i, \gamma))/(1 - f_{\text{count}}(0; x_i, \beta))$ of observing non-zero counts in hurdle models. In zero-inflation models, it is the probability $f_{\text{zero}}(0; z_i, \gamma)$ of observing a zero from the point mass component in zero-inflated models

Predicted means for the observed data can also be obtained by the `fitted()` method. Deviations between observed counts y_i and predicted means $\hat{\mu}_i$ can be obtained by the `residuals()` method returning either raw residuals $y_i - \hat{\mu}_i$ or the Pearson residuals (raw residuals standardized by square root of the variance function) with the latter being the default.

D. Replication of textbook results

Cameron and Trivedi (1998, p. 204) use a somewhat extended version of the model employed above. Because not all variables in that extended model are significant, a reduced set of variables was used throughout the main paper. Here, however, we use the full model to show that the tools in **pscl** reproduce the results of Cameron and Trivedi (1998).

After omitting the responses other than `ofp` and setting `"other"` as the reference category for `region` using

```
R> dt2 <- DebTrivedi[, -(2:6)]
R> dt2$region <- relevel(dt2$region, "other")
```

Function	Description
`print()`	simple printed display with coefficient estimates
`summary()`	standard regression output (coefficient estimates, standard errors, partial Wald tests); returns an object of class "`summary.`*class*" containing the relevant summary statistics (which has a `print()` method)
`coef()`	extract coefficients of model (full or components), a single vector of all coefficients by default
`vcov()`	associated covariance matrix (with matching names)
`predict()`	predictions (means or probabilities) for new data
`fitted()`	fitted means for observed data
`residuals()`	extract residuals (response or Pearson)
`terms()`	extract terms of model components
`model.matrix()`	extract model matrix of model components
`logLik()`	extract fitted log-likelihood
`coeftest()`	partial Wald tests of coefficients
`waldtest()`	Wald tests of nested models
`linearHypothesis()`	Wald tests of linear hypotheses
`lrtest()`	likelihood ratio tests of nested models
`AIC()`	compute information criteria (AIC, BIC, ...)

Table 3: Functions and methods for objects of class "`zeroinfl`" and "`hurdle`". The first three blocks refer to methods, the last block contains generic functions whose default methods work because of the information supplied by the methods above.

we fit a model that contains all explanatory variables, both in the count model and the zero hurdle model:

```
R> fm_hurdle2 <- hurdle(ofp ~ ., data = dt2, dist = "negbin")
```

The resulting coefficient estimates are virtually identical to those published in Cameron and Trivedi (1998, p. 204). The associated Wald statistics are also very similar provided that sandwich standard errors are used (which is not stated explicitely in Cameron and Trivedi 1998).

```
R> cfz <- coef(fm_hurdle2, model = "zero")
R> cfc <- coef(fm_hurdle2, model = "count")
R> se <- sqrt(diag(sandwich(fm_hurdle2)))
R> round(cbind(zero = cfz, zero_t = cfz/se[-seq(along = cfc)],
+   count = cfc, count_t = cfc/se[seq(along = cfc)]),
+   digits = 3)[c(3, 2, 4, 5, 7, 6, 8, 9:17, 1),]

                 zero zero_t  count count_t
healthexcellent -0.329 -2.310 -0.378  -4.312
healthpoor       0.071  0.420  0.333   5.863
numchron         0.557 10.547  0.143  10.520
```

```
adldiffyes      -0.188 -1.448  0.129   2.504
regionnoreast    0.129  1.033  0.104   1.974
regionmidwest    0.101  0.880 -0.016  -0.344
regionwest       0.202  1.509  0.123   2.444
age              0.190  2.348 -0.075  -2.339
blackyes        -0.327 -2.450  0.002   0.023
gendermale      -0.464 -4.715  0.004   0.098
marriedyes       0.247  2.379 -0.092  -2.110
school           0.054  4.109  0.022   3.824
faminc           0.007  0.365 -0.002  -0.380
employedyes     -0.012 -0.085  0.030   0.401
privinsyes       0.762  6.501  0.227   4.007
medicaidyes      0.554  3.055  0.185   2.777
(Intercept)     -1.475 -2.283  1.631   6.017
```

```
R> logLik(fm_hurdle2)
```

```
'log Lik.' -12110.49 (df=35)
```

```
R> 1/fm_hurdle2$theta
```

```
    count
0.7437966
```

There are some small and very few larger deviations in the Wald statistics which are probably explicable by different approximations to the gradient of θ (or $1/\theta$ or $\log(\theta)$) and the usage of different non-linear optimizers (and at least ten years of software development).

More replication exercises are performed in the example sections of **AER** (Zeileis and Kleiber 2008), the software package accompanying Kleiber and Zeileis (2008).

Affiliation:

Achim Zeileis
Department of Statistics
Universität Innsbruck
Universitätsstr. 15
6020 Innsbruck, Austria
E-mail: Achim.Zeileis@R-project.org
URL: http://statmath.wu-wien.ac.at/~zeileis/